

SKKU 2024 Challenge: Heisenberg Spin Glass

Donghun Jung

November 15, 2024

Part I : A first look at the Heisenberg spin glass

How many symmetry classes are there?

There are 11 distinct symmetry classes, each characterized by unique eigen-spectra. I conducted eigen-decomposition for all 64 possible combinations of the coefficients J_{ij} .

The following code generates the Heisenberg Hamiltonian $H_{\text{heisenberg}}$:

```
def __reset__(self):
    qubit_idx = [idx for idx in range(self.num_qubits)]
    combinations = itertools.combinations(qubit_idx, 2)

    H_z = np.zeros([2**self.num_qubits, 2**self.num_qubits],
                  dtype=np.complex128)
    if self.coefficients2 is not None:
        for i in range(self.num_qubits):
            temp = [self.I for _ in range(self.num_qubits)]
            temp[i] = self.Pauli_Z
            H_z += self.coefficients2[i]*self._Kron(temp)

    H_heisenberg = np.zeros([2**self.num_qubits,
                           2**self.num_qubits],
                           dtype=np.complex128)
    for coefficient, pair_idx
        in zip(self.coefficients1, combinations):
        H_heisenberg += coefficient * self._H_ij(pair_idx)
    self.H_sg = H_heisenberg + H_z
```

In this example, `self.coefficients2` is set to `None`. Within the `for` loop, each pair of qubit indices (i, j) runs through unique combinations in the following sequence: $(0, 0)$, $(0, 1)$, $(0, 2)$, $(1, 2)$, $(1, 3)$, $(2, 3)$. This setup ensures that each distinct interaction between qubits is included without repetition, covering all possible unique two-qubit interactions in the Heisenberg

Hamiltonian $H_{heisenberg}$. Additionally, the `_Kron` method is implemented to maintain the Qiskit bit-ordering convention.

```
def _Kron(self, array):
    prod = np.array(1)
    if len(array) == 0:
        return prod
    else:
        for element in array:
            prod = np.kron(element, prod)
        return prod
```

Two models are classified within the same symmetry class if their eigenspectra are identical. Here, the eigenspectrum of a Hamiltonian represents the set of its eigenvalues. To numerically determine whether two eigenspectra are equivalent, I used the `np.allclose` function.

```
symmetry_classes = {}
for coefficient_combination
    in itertools.product([1, -1], repeat=6):

    eigenspectrum = H_sg(coefficient_combination, [0, 0, 0, 0])
    symmetry_classes[coefficient_combination] = eigenspectrum

classified_groups = {}
unique_eigenspectra = {}
class_count = 0

for combination, spectrum in symmetry_classes.items():
    is_duplicate = False
    for class_id, class_spectrum in unique_eigenspectra.items():
        if np.allclose(class_spectrum, spectrum._eigenSpectrum()):
            is_duplicate = True
            classified_groups[class_id].append(combination)
            break

    if is_duplicate:
        continue
    else:
        class_count += 1
        classified_groups[class_count] = [combination]
        unique_eigenspectra[class_count] = spectrum._eigenSpectrum()
```

Detailed information on eigenspectra and coefficient values can be found in the next questions.

For each symmetry class, provide one set of J_{ij} coefficients for a model in that symmetry class.

The Heisenberg Hamiltonian $\mathcal{H}_{\text{heisenberg}}$ is defined as:

$$\mathcal{H}_{\text{heisenberg}} = \sum_{(i,j)} J_{i,j} H_{i,j} \quad (1)$$

where

$$H_{i,j} = \sigma_x^i \sigma_x^j + \sigma_y^i \sigma_y^j + \sigma_z^i \sigma_z^j \quad (2)$$

The interaction is ferromagnetic when $J_{ij} = 1$ and anti-ferromagnetic when $J_{ij} = -1$. This Hamiltonian pocesses intrinsic symmetry: Interchange symmetry. Under the interchange interaction on qubit i and j , both of Hamiltonian eigenvalues and eigenstates remain unchanged. Therefore, the elements in the same symmetry class can be generated via interchange operation.

The blue edge denotes ferromagnetic interaction and red edge denotes anti-ferromagnetic interaction.

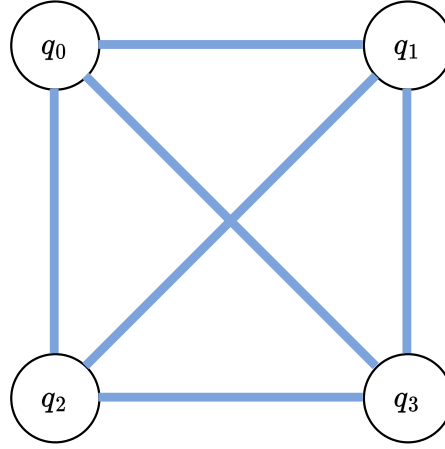


Figure 1: caption

J_{01}	J_{02}	J_{03}	J_{12}	J_{13}	J_{23}
1	1	1	1	1	1

Table 1: Example LaTeX Table

Class 1. The whole interactions are ferromagnetic. There is no other possible element even by using interchange interaction.

Class 2

J_{01}	J_{02}	J_{03}	J_{12}	J_{13}	J_{23}
1	1	1	1	1	-1
1	1	1	1	-1	1
1	1	1	-1	1	1
1	1	-1	1	1	1
1	-1	1	1	1	1
-1	1	1	1	1	1

Table 2: Example LaTeX Table

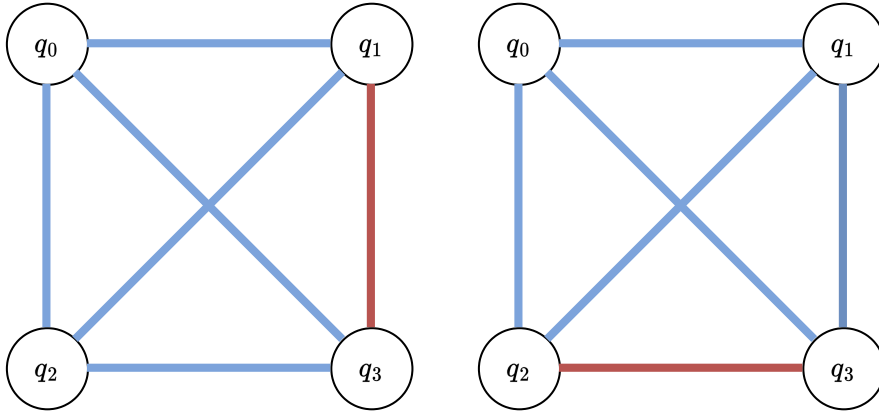


Figure 2: caption

Class 2. Only one interaction term is antiferromagnetic. By using interchange operation, we can derive 6 elements as the same symmetry class. For example, from the left one in the figure with coefficients $(1, 1, 1, 1, -1, 1)$, we can derive right one with coefficients $(1, 1, 1, 1, 1, -1)$ by using interchange operation on qubit 1 and qubit 2. The other models can be derived by applying one or multiple interchange operations.

Class 3

Class 3. Two interaction terms are antiferromagnetic. As shown in the figure, one qubit has two antiferromagnetic interaction. By using interchange operation, we can derive 12 elements as the same symmetry class. For example, from the left one in the figure with coefficients $(1, 1, 1, 1, -1, -1)$, we

J_{01}	J_{02}	J_{03}	J_{12}	J_{13}	J_{23}
1	1	1	1	-1	-1
1	1	1	-1	1	-1
1	1	1	-1	-1	1
1	1	-1	1	1	-1
1	1	-1	1	-1	1
1	-1	1	1	1	-1
1	-1	1	-1	1	1
1	-1	-1	1	1	1
-1	1	1	1	-1	1
-1	1	1	-1	1	1
-1	1	-1	1	1	1
-1	-1	1	1	1	1

Table 3: Example LaTeX Table

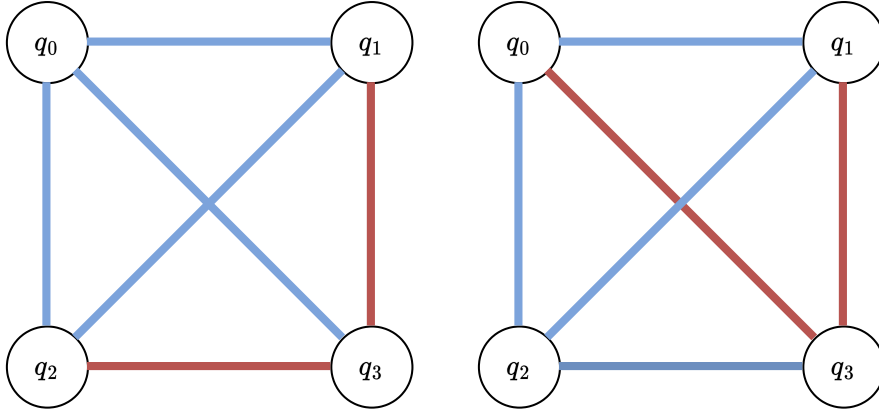


Figure 3: caption

can derive right one with coefficients $(1, 1, -1, 1, -1, 1)$ by using interchange operation on qubit 0 and qubit 2. The other models can be derived by applying one or multiple interchange operations.

Class 4

Class 3. Three interaction terms are antiferromagnetic. As shown in the figure, three qubits have two antiferromagnetic interactions while the remaining qubit has only ferromagnetic interaction. By using interchange operation, we can derive 4 elements as the same symmetry class. For example, from the left one in the figure with coefficients $(1, 1, 1, -1, -1, -1)$, we can derive right one with coefficients $(-1, 1, -1, 1, -1, 1)$ by using inter-

J_{01}	J_{02}	J_{03}	J_{12}	J_{13}	J_{23}
1	1	1	-1	-1	-1
1	-1	-1	1	1	-1
-1	1	-1	1	-1	1
-1	-1	1	-1	1	1

Table 4: Example LaTeX Table

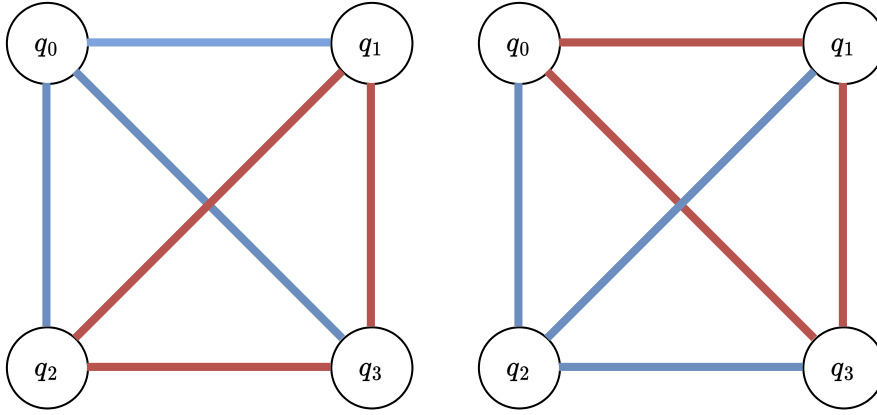


Figure 4: caption

change operation on qubit 0 and qubit 2. The other models can be derived by applying one or multiple interchange operations.

Class 5

J_{01}	J_{02}	J_{03}	J_{12}	J_{13}	J_{23}
1	1	-1	1	-1	-1
1	-1	1	-1	1	-1
-1	1	1	-1	-1	1
-1	-1	-1	1	1	1

Table 5: Example LaTeX Table

Class 5. Three interaction terms are antiferromagnetic. As shown in the figure, one qubit has only antiferromagnetic interactions. By using interchange operation, we can derive 4 elements as the same symmetry class. For example, from the left one in the figure with coefficients $(1, 1, -1, 1, -1, -1)$,

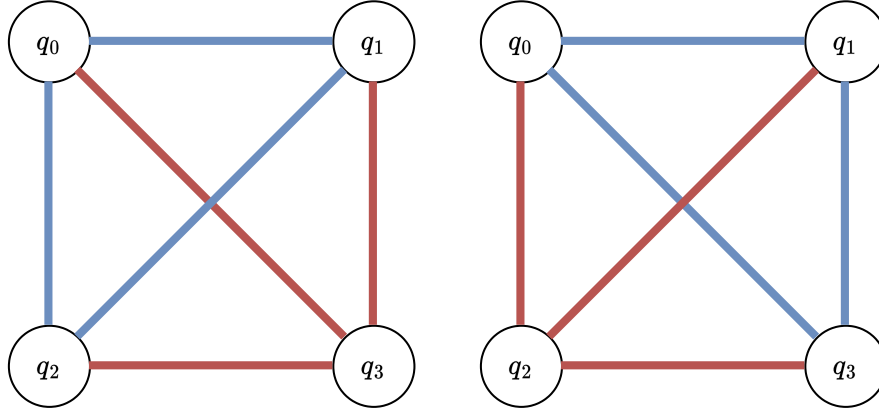


Figure 5: caption

we can derive right one with coefficients $(1, -1, 1, 1, -1, -1)$ by using interchange operation on qubit 2 and qubit 3. The other models can be derived by applying one or multiple interchange operations.

Class 6

J_{01}	J_{02}	J_{03}	J_{12}	J_{13}	J_{23}
1	1	-1	-1	1	1
1	-1	1	1	-1	1
-1	1	1	1	1	-1

Table 6: Example LaTeX Table

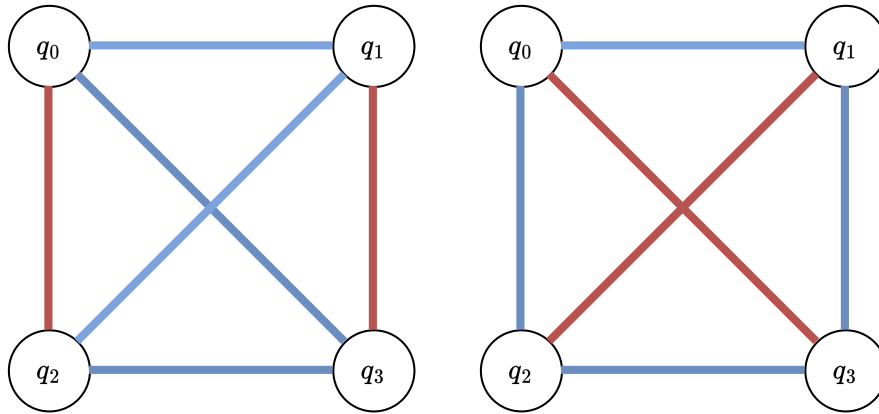


Figure 6: caption

Class 6. Two interaction terms are antiferromagnetic. As shown in the figure, all qubit has only one antiferromagnetic interaction unlike class 2 symmetry group that one qubit has two antiferromagnetic interaction. By using interchange operation, we can derive 3 elements as the same symmetry class. For example, from the left one in the figure with coefficients $(1, -1, 1, 1, -1, 1)$, we can derive right one with coefficients $(1, 1, -1, -1, 1, 1)$ by using interchange operation on qubit 0 and qubit 1. The other model can be derived by applying one or multiple interchange operations.

Class 7

J_{01}	J_{02}	J_{03}	J_{12}	J_{13}	J_{23}
1	1	-1	-1	1	-1
1	1	-1	-1	-1	1
1	-1	1	1	-1	-1
1	-1	1	-1	-1	1
1	-1	-1	1	-1	1
1	-1	-1	-1	1	1
-1	1	1	1	-1	-1
-1	1	1	-1	1	-1
-1	1	-1	1	1	-1
-1	1	-1	-1	1	1
-1	-1	1	1	1	-1
-1	-1	1	1	-1	1

Table 7: Example LaTeX Table

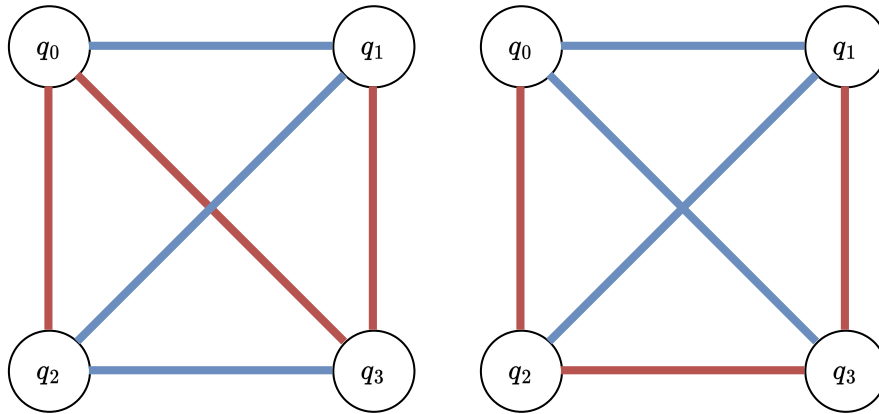


Figure 7: caption

Class 7. Three interaction terms are antiferromagnetic. As shown in the figure, each qubit has one or two antiferromagnetic interactions. In this case, this symmetry class also possesses sign-flip symmetry. After the sign flip operation, the Hamiltonian is still in the same class. By using interchange operation and sign flip operation, we can derive 12 elements as the same symmetry class. For example, from the left one in the figure with coefficients $(1, -1, -1, 1, -1, 1)$, we can derive right one with coefficients $(1, -1, 1, 1, -1, -1)$ by using interchange operation on qubit 0 and qubit 2. The other model can be derived by applying one or multiple interchange operations and sign flip operation.

Class 8

J_{01}	J_{02}	J_{03}	J_{12}	J_{13}	J_{23}
1	1	-1	-1	-1	-1
1	-1	1	-1	-1	-1
1	-1	-1	1	-1	-1
1	-1	-1	-1	1	-1
-1	1	1	-1	-1	-1
-1	1	-1	1	-1	-1
-1	1	-1	-1	-1	1
-1	-1	1	-1	1	-1
-1	-1	1	-1	-1	1
-1	-1	-1	1	1	-1
-1	-1	-1	1	-1	1
-1	-1	-1	-1	1	1

Table 8: Example LaTeX Table

Class 8. From now on, it is convenient to focus on ferromagnetic interaction. Two interaction terms are ferromagnetic. As shown in the figure, one qubit has two antiferromagnetic interaction. By using interchange operation, we can derive 12 elements as the same symmetry class. For example, from the left one in the figure with coefficients $(-1, -1, -1, -1, 1, 1)$, we can derive right one with coefficients $(-1, -1, 1, -1, 1, -1)$ by using interchange operation on qubit 0 and qubit 2. The other models can be derived by applying one or multiple interchange operations. Note that this symmetry class can also be derived by sign flip operation on symmetry class 3.

Class 9

Class 9. Two interaction terms are ferromagnetic. As shown in the figure, each qubit has only one antiferromagnetic interaction. By using interchange

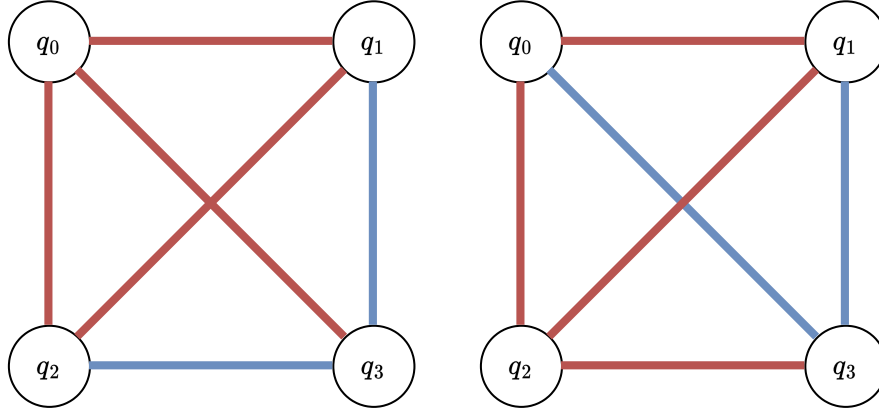


Figure 8: caption

J_{01}	J_{02}	J_{03}	J_{12}	J_{13}	J_{23}
1	-1	-1	-1	-1	1
-1	1	-1	-1	1	-1
-1	-1	1	1	-1	-1

Table 9: Example LaTeX Table

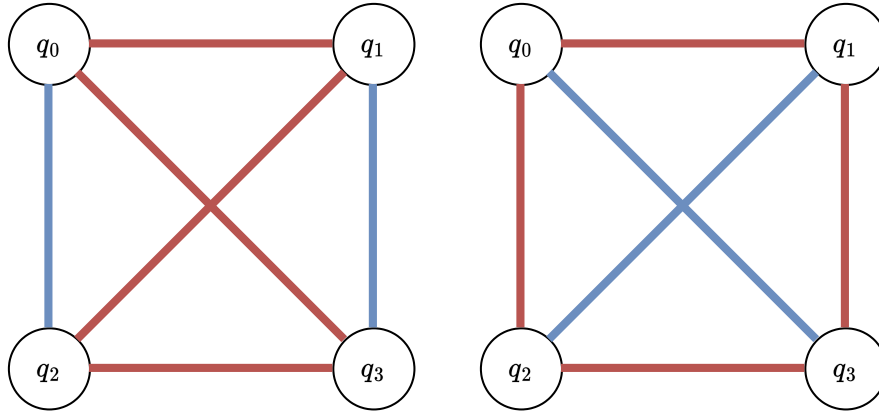


Figure 9: caption

operation, we can derive 3 elements as the same symmetry class. For example, from the left one in the figure with coefficients $(-1, 1, -1, -1, 1, -1)$, we can derive right one with coefficients $(-1, -1, 1, 1, -1, -1)$ by using interchange operation on qubit 0 and qubit 1. The other model can be derived by applying one or multiple interchange operations. Note that this symmetry

class can also derived by sign flip operation on symmetry class 6.

Class 10

J_{01}	J_{02}	J_{03}	J_{12}	J_{13}	J_{23}
1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1
-1	-1	1	-1	-1	-1
-1	-1	-1	1	-1	-1
-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	1

Table 10: Example LaTeX Table

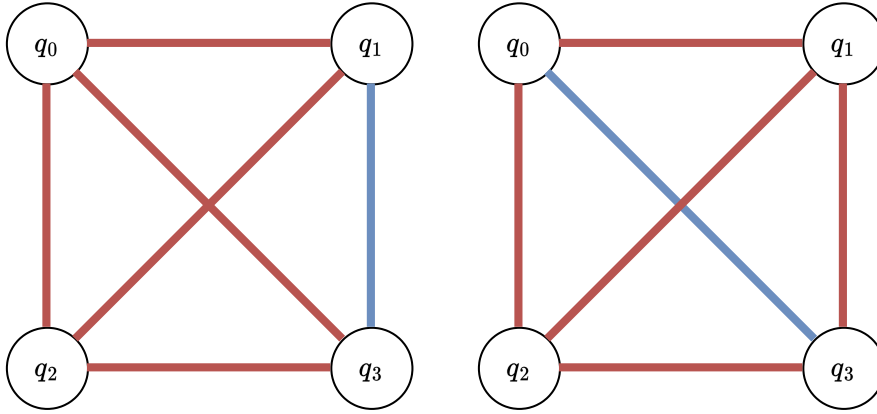


Figure 10: caption

Class 10. Only one interaction term is ferromagnetic. By using interchange operation, we can derive 6 elements as the same symmetry class. For example, from the left one in the figure with coefficients $(-1, -1, -1, -1, 1, -1)$, we can derive right one with coefficients $(-1, -1, 1, -1, -1, -1)$ by using interchange operation on qubit 0 and qubit 1. The other model can be derived by applying one or multiple interchange operations. Note that this symmetry class can also derived by sign flip operation on symmetry class 2.

Class 11

Class 11. The whole interactions are antiferromagnetic. There is no other possible element even by using interchange interaction. This symmetry class can be derived from sign flip operation on symmetry class 1.

J_{01}	J_{02}	J_{03}	J_{12}	J_{13}	J_{23}
-1	-1	-1	-1	-1	-1

Table 11: Example LaTeX Table

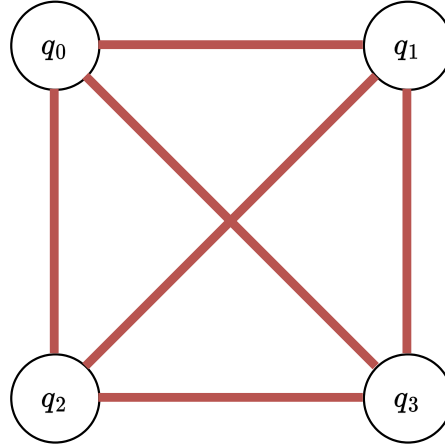


Figure 11: caption

What is the eigenspectrum of each symmetry class?

Do any of the symmetry classes have an eigenspectrum that is sign invariant? In other words, if you change the sign of each eigenvalue, does the overall set of eigenvalues remain unchanged?

As shown in Figure , the eigenspectrum of class 7 has sign flip invariance. It is because the Hamiltonian of class 7 is still in the class 7 even after sign flip operation.

Part 2 : Adiabatic evolution to the ground state

Plot the eigenspectra as a function of s for each symmetry class.

Class 1

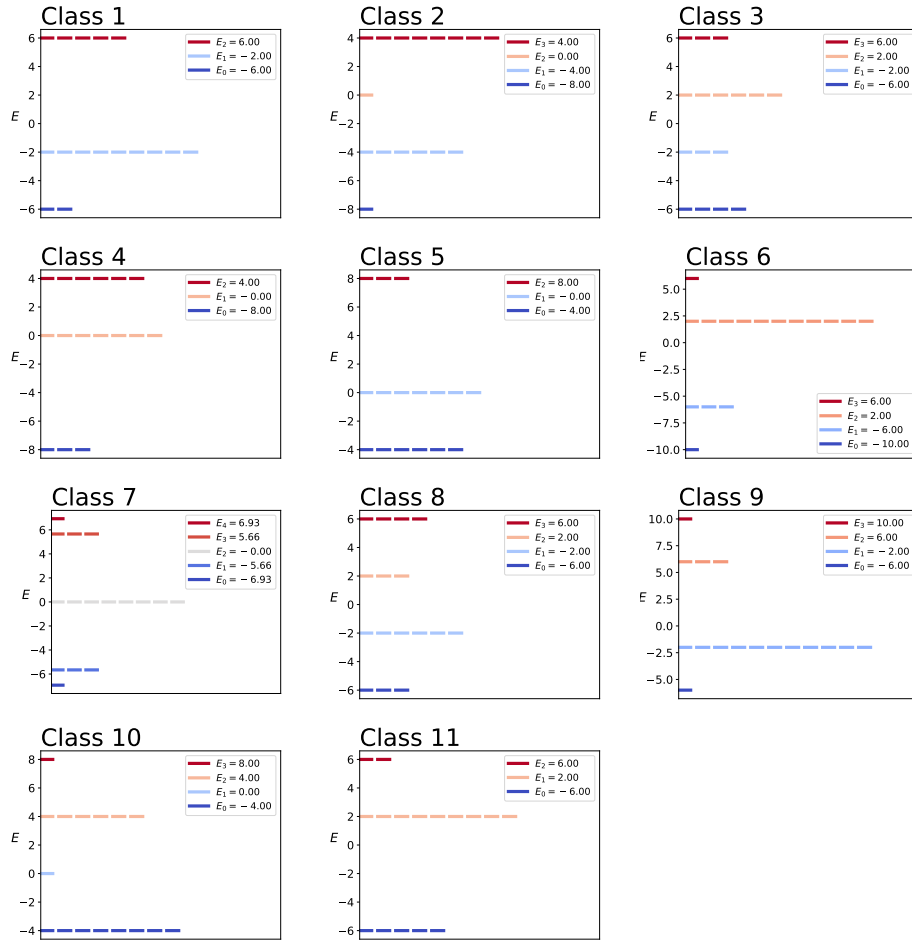


Figure 12: caption

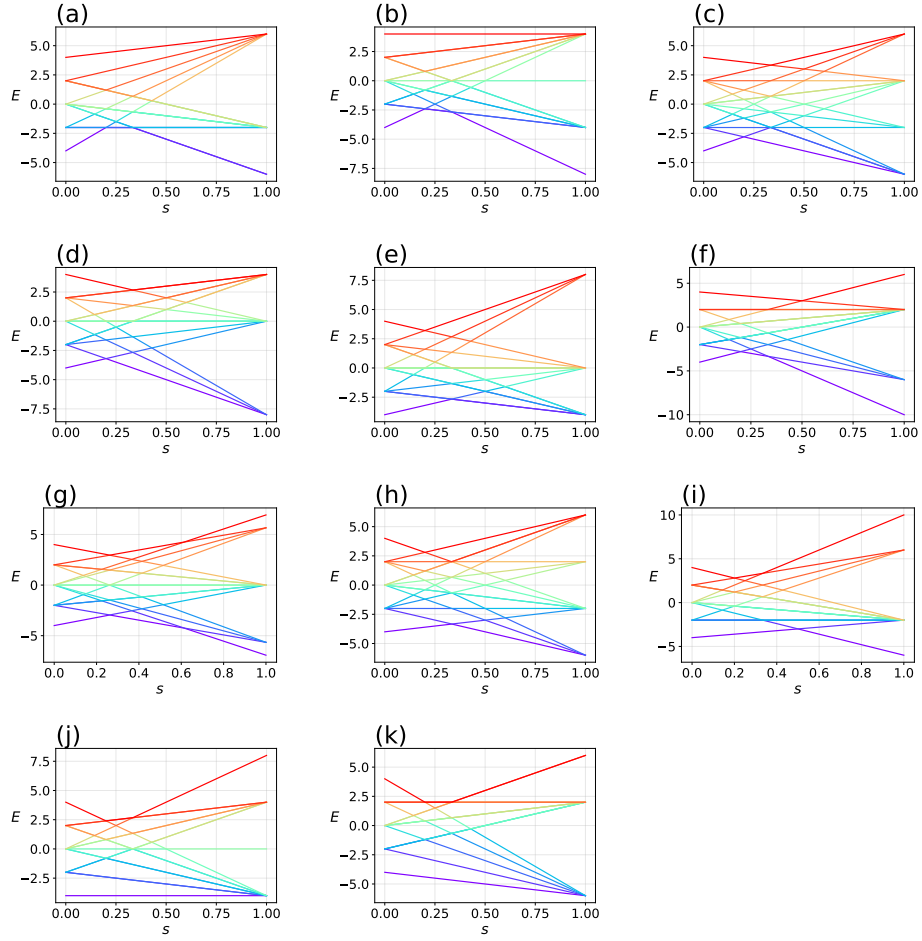


Figure 13: caption

Plot the eigenspectrum for H_{sg} as a function of s .

Write a program to carry out adiabatic evolution, starting from the pure mixer Hamiltonian and ending with H_{sg} . Two parameters govern this program: the time τ over which the adiabatic evolution takes place and the number of time steps N .

```
def circuit(self, parameters):
    qc = qiskit.QuantumCircuit(self.num_qubits)
    for i in range(1, self.num_qubits):
        qc.x(i)
    for pair, parameter in zip(self.index, parameters):
```

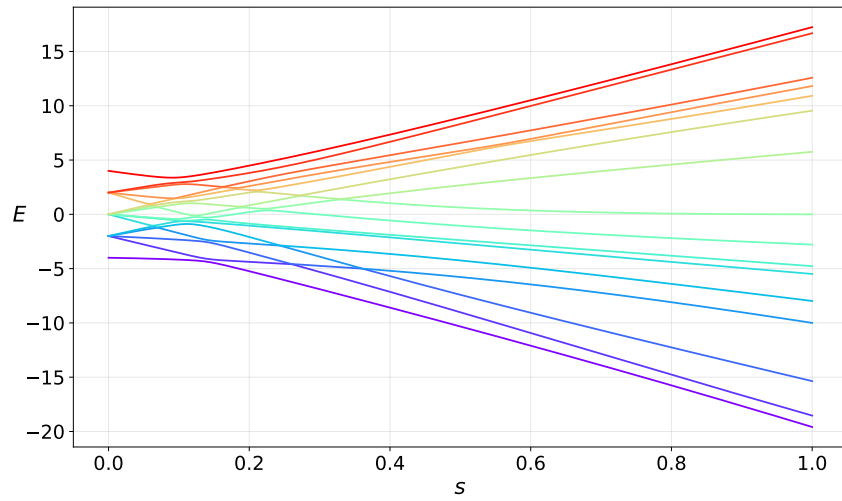


Figure 14

```
qc.rxx(parameter, pair[0], pair[1])
qc.ryy(parameter, pair[0], pair[1])

qc.rz(np.pi/2, 0)
temp = self.ansatz(parameters)[[7,11,13,14]]

temp[0:3] *= (1j if temp[3] > 0 else -1j)
temp = (temp > 0)
for i in range(3):
    qc.rz(np.pi, 3-i) if temp[i] else None

self.qc = qc

return qc
```

In order to match the target vector of sixteen probabilities by performing adiabatic evolution, what value do you need to use for τ and N ? The exact target probabilities and the probabilities resulting from adiabatic evolution should match to three significant decimal digits.

The required running time for adiabatic evolution is known to the eigenspectrum of H_s , in particular the energy gap, δ , between the two lowest energy

eigenvalues. The time for adiabatic evolution must obey:

$$\tau \gg \frac{\epsilon}{\Delta^2} \quad (3)$$

where $\Delta = \min_{0 \leq s \leq 1} (E_1(s) - E_0(s))$ and ϵ is the largest value of $H_P - H_0$. In our case, $\Delta \simeq 0.2666$ and $\epsilon \simeq 17.79$, then, we have $\tau \simeq 250.4$. Then, I tuned τ and the number of step N , which is determined by dt , I found that $\tau = 1000$ and $N = 2000$ yield our desired result.

List out the terms in H_{sg} and H_m and, for each term, provide the gate that arises when that term is exponentiated. You can use any gate in Qiskit's standard gate set. How many single-qubit gates are needed for each time step? How many two-qubit gates are needed for each time step?

Heisenberg Spin Glass

Assess the circuit needed to carry out the adiabatic evolution. Roughly how many single-qubit gates and two-qubit gates would be required to achieve three-digit accuracy for the sixteen probabilities characterizing the ground state? Do you think this is practical?

It seems that the adiabatic evolution is not that practical.

Part 3 : A simpler path to the ground state

Build a numerical program as outlined above, which contains arbitrary angle parameters and qubit pairs for XY-mixers. Experiment with the XY-mixers to obtain angle parameters and qubit pairs that rotate the final state so that its probabilities match the ground state probabilities of H_{sg} . This problem can be solved using three XY-mixers.

I used ansatz the state, initialized to $|1110\rangle$ first and three XY-mixer acting on qubit-0 and -1, qubit-0 and -2, and qubit-0 and -3. The following code performs the same operation:

```
def ansatz(self, parameters):
    if len(self.index) != len(parameters):
        raise ValueError("Length should be matched")

    state = np.zeros(16, dtype=np.complex64)
    state[14] = 1
```



```

operation = [self._XY_mixer(pair, parameter)
              for pair, parameter
              in zip(self.index, parameters)]
for op in operation:
    state = op @ state

return state

```

Convert the numerical program into a quantum circuit. Verify that the output of this quantum circuit generates the correct probabilities for the ground state of H_{sg} .

Noting that $[\sigma_x^i \sigma_x^j, \sigma_y^i \sigma_y^j] = 0$ the XY-mixer can be decomposed into $e^{-i\frac{\theta}{2}(\sigma_x^i \sigma_x^j + \sigma_y^i \sigma_y^j)} = e^{-i\frac{\theta}{2}\sigma_x^i \sigma_x^j} e^{-i\frac{\theta}{2}\sigma_y^i \sigma_y^j}$, that is, a combination of RXX gate and RYY gate. Then, our quantum circuit can be implemented (i) apply X gate on qubit 1, 2, 3, (ii) apply RXX and RYY gate on (0,1), (0,2), (0,3) qubits. This will yield a state that produces the similar signal distribution with the exact ground state. The code is implemented as:

```

def circuit(self, parameters):
    qc = qiskit.QuantumCircuit(self.num_qubits)
    for i in range(1, self.num_qubits):
        qc.x(i)
    for pair, parameter in zip(self.index, parameters):
        qc.rxx(parameter, pair[0], pair[1])
        qc.ryy(parameter, pair[0], pair[1])

    return qc

```

Check the phases of the non-zero components of the quantum circuit created in 3.b. Are they all real? If so, you can skip this step. Otherwise, find circuit elements that you can add to the end of your circuit so that all non-zero computational basis states have the same phase.

The exact ground state has only real coefficients whose sign is $(+, +, +, -)$ in a order of $|0111\rangle, |1011\rangle, |1101\rangle, |1110\rangle$. However, the circuit shown above generates a state whose coefficient of $ket1110$ is real while other coefficients

are pure imaginary.

$$e^{-i\frac{\theta_3}{2}(\sigma_x^0\sigma_x^3+\sigma_y^0\sigma_y^3)}e^{-i\frac{\theta_2}{2}(\sigma_x^0\sigma_x^2+\sigma_y^0\sigma_y^2)}e^{-i\frac{\theta_1}{2}(\sigma_x^0\sigma_x^1+\sigma_y^0\sigma_y^1)}|1110\rangle = -i\cos\theta_1\cos\theta_2\sin\theta_3|0111\rangle + i\cos\theta_1\sin\theta_2\cos\theta_3|1011\rangle - i\sin\theta_1\cos\theta_2\cos\theta_3|1101\rangle + i\sin\theta_1\sin\theta_2\cos\theta_3|1110\rangle \quad (4)$$

For convenience, let me denote it as a vector $(-i\cos\theta_1\cos\theta_2\sin\theta_3, i\cos\theta_1\sin\theta_2\cos\theta_3, -i\sin\theta_1\cos\theta_2\cos\theta_3, i\sin\theta_1\sin\theta_2\cos\theta_3)$. After applying RZ gate on qubit 0 with an angle $\frac{\pi}{2}$, to make imaginary terms to real. Then next we should tune the sign of each term to be $(+, +, +, -)$ or $(-, -, -, +)$. Noting that a RZ gate with angle π has effect that no sign change when the state of target qubit is 0, sign change otherwise. The solution is quite simple. If the sign of a term is differ from the one of $|1110\rangle$, there is not any required additional operations. Even, there is another $R_z(\pi)$ operation on the other qubit there is no other change on that sign relation. Therefore, if the sign of the coefficients of a basis whose 0 state is on i qubit is same with the one of $|1110\rangle$, apply $R_z\pi$ on qubit i . This will results in correct ground state. The full code implementation including using XY-mixer and phase adjustment is below:

```
def circuit(self, parameters):
    qc = qiskit.QuantumCircuit(self.num_qubits)
    for i in range(1,self.num_qubits):
        qc.x(i)
    for pair, parameter in zip(self.index, parameters):
        qc.rxx(parameter, pair[0], pair[1])
        qc.ryy(parameter, pair[0], pair[1])

    qc.rz(np.pi/2, 0)
    temp = self.ansatz(parameters)[[7,11,13,14]]

    temp[0:3] *= (1j if temp[3] > 0 else -1j)
    temp = (temp > 0)
    for i in range(3):
        qc.rz(np.pi, 3-i) if temp[i] else None

    self.qc = qc

    return qc
```

Use your quantum circuit to measure the expectation value of the ground state of H_{sg} . You will have to repeat your circuit three times - once to measure in each of the X, Y and Z computational bases. Each term of H_{sg} can be measured in one of the three computational bases. Sum the contributions from the three bases and confirm that the summed energy is equal to the ground state eigenvalue

Trivially, the expectation value of the system Hamiltonian H_{sg} is sum of the expectation value of its components. Here, I gonna propose a general methodology to extract expectation value of σ_z^i and $\sigma_z^i \sigma_z^j$. First, σ_z^i can be decomposed into:

$$\sigma_z^i = I_2^{\otimes 3-i} \otimes \sigma_z \otimes I_2^{\otimes i} \quad (5)$$

$$= (|0\rangle\langle 0| + |1\rangle\langle 1|)^{\otimes 3-i} \otimes (|0\rangle\langle 0| - |1\rangle\langle 1|) \otimes (|0\rangle\langle 0| + |1\rangle\langle 1|)^{\otimes i} \quad (6)$$

$$(7)$$

where I_2 is 2×2 identity matrix. Then the expectation value of σ_z^i for a given state is:

$$\langle \sigma_z^i \rangle = \sum_{(b_i=0)} m_b - \sum_{(b_i=1)} m_b \quad (8)$$

where b is the bit string, $b = b_3 b_2 b_1 b_0$ and m_b is the estimated probability with bit string b , $m_b = |\langle b | \psi \rangle|^2$. Second, $\sigma_z^i \sigma_z^j$ is decomposed into:

$$\sigma_z^i \sigma_z^j = I_2^{\otimes 3-j} \otimes \sigma_z \otimes I_2^{\otimes i-j-1} \otimes \sigma_z \otimes I_2^{\otimes i} \quad (9)$$

$$= (|0\rangle\langle 0| + |1\rangle\langle 1|)^{\otimes 3-i} \otimes (|0\rangle\langle 0| - |1\rangle\langle 1|) \otimes (|0\rangle\langle 0| + |1\rangle\langle 1|)^{\otimes i-j-1} \otimes (|0\rangle\langle 0| - |1\rangle\langle 1|) \otimes (|0\rangle\langle 0| + |1\rangle\langle 1|)^{\otimes i} \quad (10)$$

$$(11)$$

where $i > j$ is supposed without loss of generality. Then, the expectation value of $\sigma_z^i \sigma_z^j$ for a given state is:

$$\langle \sigma_z^i \sigma_z^j \rangle = \sum_{(b_i b_j=0)} m_b - \sum_{(b_i b_j=1)} m_b \quad (12)$$

Here, $\langle \sigma_x^i \sigma_x^j \rangle$ and $\langle \sigma_y^i \sigma_y^j \rangle$ can be estimated in the same way by change 0 to $+, i$ and 1 to $-, -i$ for each. The following code performs this expectation value extraction:

```
def _calculate_expectation(self, basis, i, j=None):
    expectation_value = 0
    measurement_outcome = self.measurement_outcome[basis]
```

```

for bitstring, probability in measurement_outcome.items():
    bit_i = (int(bitstring, 2) >> i) & 1
    if j is not None:
        bit_j = (int(bitstring, 2) >> j) & 1
        sign = 1 if (bit_i ^ bit_j) == 0 else -1
    else:
        sign = 1 if bit_i == 0 else -1

    expectation_value += sign * probability

return expectation_value

```

As discussed above, the measurement should be performed on x, y, z basis. Generally, the measurement is performed on z basis, which are described as POVM set, $\{|0000\rangle\langle 0000|, |0001\rangle\langle 0001|, \dots, |1111\rangle\langle 1111|\}$. By applying additional unitary operation before the measurement, we can make it as if we are performing measurement on a different basis. In other words, for a certain POVM operator Ω , we apply additional unitary operation U right before the measurement, we are performing measurement $U^\dagger \Omega U$. If $U = H$, we can perform measurement on x -basis. And $U = HS$, we can perform measurement on y -basis. The following code generates such three circuits performing measurement on x, y, z basis:

```

def perform_measure(self, num_shots=10000):
    qc_x = self.qc.copy()
    qc_y = self.qc.copy()
    qc_z = self.qc.copy()

    for i in range(self.num_qubits):
        qc_x.h(i)
        qc_y.s(i)
        qc_y.h(i)

    qc_x.measure_all()
    qc_y.measure_all()
    qc_z.measure_all()

    outcome_x = MeasurementResultHandler(backend.run(qc_x, shots=num_shots).result())
    outcome_y = MeasurementResultHandler(backend.run(qc_y, shots=num_shots).result())
    outcome_z = MeasurementResultHandler(backend.run(qc_z, shots=num_shots).result())

    self.measurement_outcome = {

```

```

        "x" : outcome_x,
        "y" : outcome_y,
        "z" : outcome_z,
    }

    return self.measurement_outcome

```

where MeasurementResultHandler converts measurement shots into estimated probability and fill zero shot results as zero probability.

```

def MeasurementResultHandler(result:dict, num_qubits:int, num_shots:int):
    new_result = {}
    for i in range(2**num_qubits):
        temp = format(i,"#b")[2:]
        key = "".join(["0" for _ in range(num_qubits - len(temp))]) + temp
        new_result[key] = np.array(result[format(i,"#x")])/num_shots if res
    return new_result

```